Software Requirements

User & System Requirements,
Functional vs Non-Functional,
Requirements Engineering Activities

User & System Requirements

- User Requirements: High-level descriptions of system functions
- System Requirements: Detailed descriptions of system functionality
- Written differently to support both stakeholders and developers

User & System Requirements

Requirement Type	Definition	Written For	Level of Detail
User Requirements	High-level statements describing what the user wants the system to do	Non-technical stakeholders	Simple, natural language
System Requirements	Detailed, technical specifications describing system functionality	Developers, engineers	Precise and structured

Project Example: **Online Food Ordering System**

User Requirements

Written in **simple language**, focused on goals:

1.Users should be able to browse restaurants and menus.

2.Users should be able to place a food order online.

3.Users should be able to pay securely.

4.Users should receive order status updates.

Project Example: **Online Food Ordering System**

System Requirements (technical, measurable, implementable)

User Requirement	System Requirement Examples
Browse restaurants & menus	SR1: The system shall display restaurant list within 3 seconds. SR2: The system shall allow filtering by location and cuisine.
Place orders online	SR3: The system shall store order details in the database. SR4: The system shall send order details to the restaurant system automatically.
Secure payment	SR5: The system shall integrate with SSL encryption. SR6: The system shall support payments via card, mobile banking, and cash-on-delivery.
Order updates	SR7: The system shall send notifications via SMS and app alerts. SR8: The system shall allow real-time tracking from restaurant to delivery.

Functional & Non-Functional Requirements

Functional Requirements

- Define specific functions or behaviors of the system.
- Focus on **what the system should do**.
- Directly related to user tasks or features.
- Example: *“The system shall allow users to register and log in.”*

Non-Functional Requirements

- Define quality attributes or constraints.
- Focus on how the system performs functions.
- Related to system qualities such as performance, security, or usability.
- Example: “The system shall load each page within 3 seconds.

Project Example — Online Food Ordering System

Functional Requirements:

- The system must allow users to browse restaurants and menus.
- The system must let users place orders online.
- The system must process secure payments.
- The system must send confirmation messages after payment.

Non-Functional Requirements:

- The system should load any page within **3 seconds**.
- The system must ensure **99.9% uptime**.
- The system should handle at least **500 concurrent users**.
- The system should be **accessible from mobile and desktop devices**.

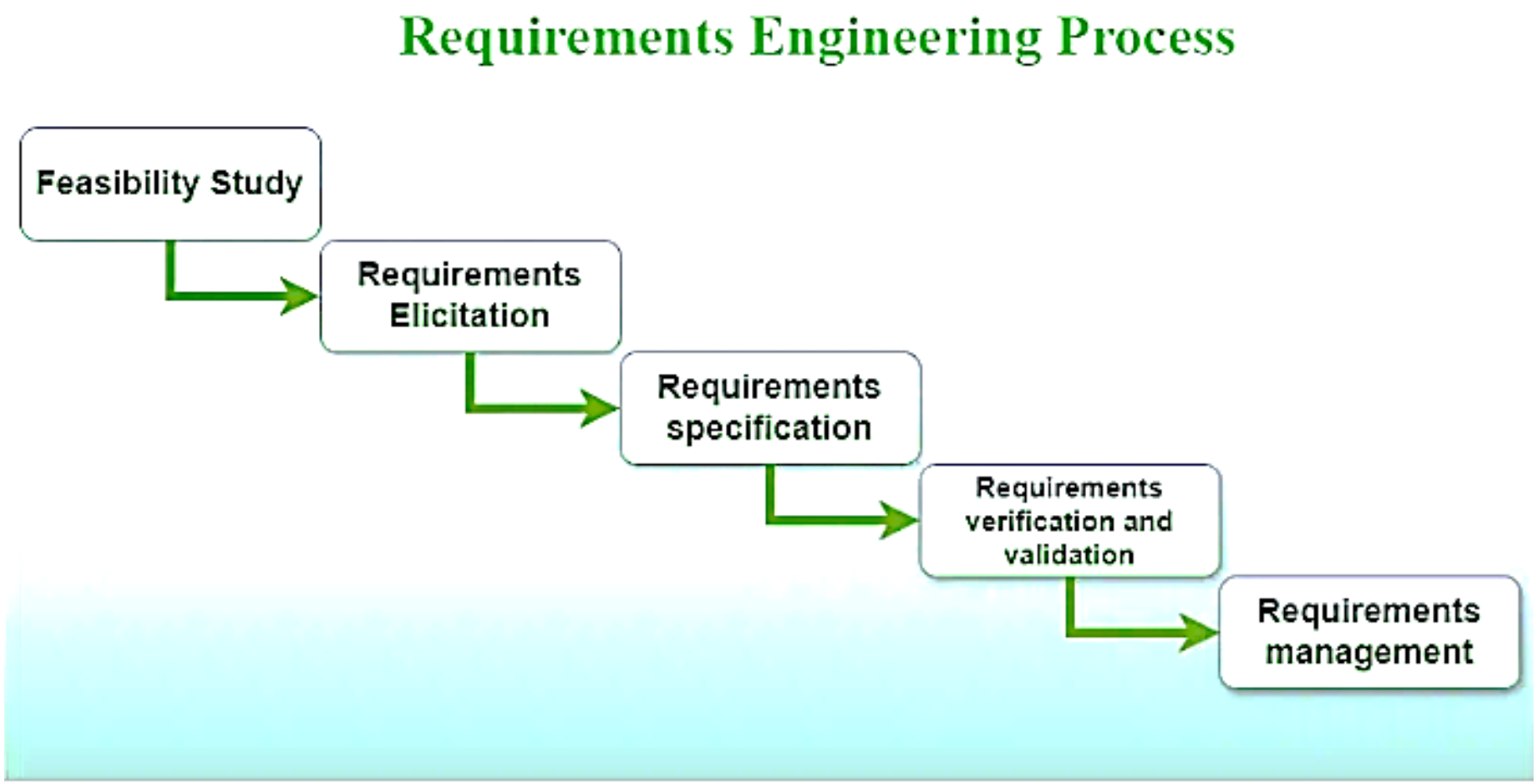
Differences Between Functional & Non-Functional Requirements

Aspect	Functional Requirements	Non-Functional Requirements
Definition	Define what the system should do	Define how the system should perform
Focus	System features and behaviors	Quality, performance, constraints
Type of Requirement	Action-oriented	Quality-oriented
Example	“User can log in to their account.”	“Login should take less than 2 seconds.”
Testing	Verified by functionality testing	Verified by performance or usability testing
Purpose	To ensure correct system operation	To ensure system efficiency and user satisfaction

Requirements Engineering Activities

- A disciplined process of **discovering**, **documenting**, and **maintaining** software requirements
- Ensures the system meets user needs and business goals
- Involves continuous communication with stakeholders
- Prevents misunderstandings and costly rework
- Ensures quality and customer satisfaction
- Creates a clear roadmap for development
- Reduces project risks and scope creep

Key Activities in Requirements Engineering



Requirements Elicitation

Definition:

Process of discovering requirements from stakeholders

Techniques:

- Interviews
- Surveys & Questionnaires
- Observation
- Brainstorming
- Use cases & Prototyping

Goal: Understand stakeholder needs clearly

Requirements Analysis

Definition:

Examining requirements for conflicts, completeness, and feasibility

Tasks include:

- Prioritizing requirements
- Resolving conflicts among stakeholders
- Modeling requirements (e.g., diagrams)
- Transforming user requirements into system requirements

Goal: Ensure requirements are correct and detailed

Requirements Validation

Definition:

Ensuring the documented requirements **accurately** represent stakeholder needs

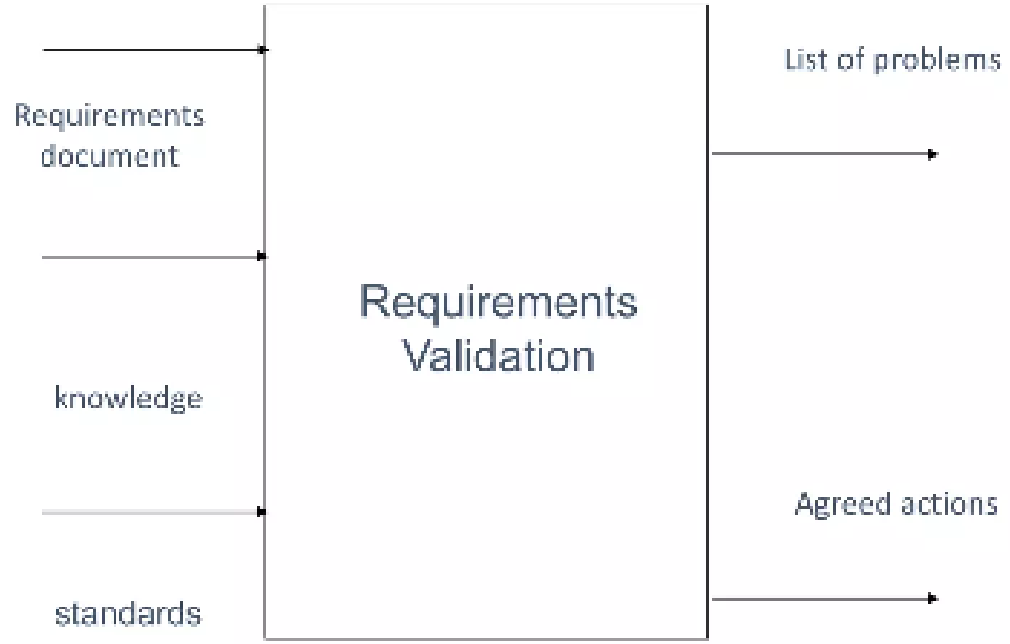
Validation techniques:

- Reviews & inspections
- Prototyping
- Test case generation
- Requirement traceability checks

Goal: Build the *right* system

Requirements Validation

Validation Inputs and Outputs



Relationships Between Activities

- Elicitation informs analysis
- Analysis identifies conflicts and improvements
- Validation ensures accuracy before implementation
- Activities are iterative and interconnected

Example User Stories

Identifier	User Story	Size
ST-1	As an authorized person (tenant or landlord), I can keep the doors locked at all times.	4 poin
ST-2	As an authorized person (tenant or landlord), I can lock the doors on demand.	3 pts
ST-3	The lock should be automatically locked after a defined period of time.	6 pts
ST-4	As an authorized person (tenant or landlord), I can unlock the doors. (Test: Allow a small number of mistakes, say three.)	9 poin
ST-5	As a landlord, I can at runtime manage authorized persons.	10 pt
ST-6	As an authorized person (tenant or landlord), I can view past accesses.	6 pts
ST-7	As a tenant, I can configure the preferences for activation of various devices.	6 pts
ST-8	As a tenant, I can file complaint about “suspicious” accesses.	6 pts